

RAM Module UVM Verification

Design Analysis:

1. Bugs found in design

- Case (2'b11): changed to read address
- Expanded the condition during the assignment for tx to avoid conflict with rst_n condition, so now it also checks the rst_n.

2. Single Port Ram to be used as golden model

The golden model established in design, was the single port ram designed in the project for the design part of the diploma.

3. Verification Plan

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
REQ_1	Verify reset functionality — all outputs and addresses should be cleared when rst_n = 0.	Drive rst_n = 0 while toggling clk, randomize din, rx_valid values, then deassert reset.	Coverpoint on rst_n transition 1 → 0 → 1. Check bins for dout, tx_valid being 0 after reset.	Scoreboard compares DUT and golden model after reset; all outputs must match initial state (zeros).
REQ_2	Verify write address load operation (din[9:8] = 2'b00).	Generate transactions with din[9:8]=2'b00 and randomized lower bits [7:0] (address).	Coverpoint on din[9:8] with bins {00,01,10,11}. Cross din[9:8] with rx_valid=1.	Check scoreboard/golden model receives same address; DUT must store new write address correctly.
REQ_3	Verify write data operation (din[9:8] = 2'b01) writes data to previously stored address.	Generate sequence of write address followed by write data with randomized data [7:0].	Cover ordering: write_addr → write_data. Coverpoint for number of unique addresses written.	Compare memory contents (MEM[addr]) in DUT vs golden model after each write transaction.
REQ_4	Verify read address load operation (din[9:8] = 2'b10).	Generate transactions setting read address using random [7:0] values.	Coverpoint for din[9:8]=2'b10 and unique address reads.	Ensure next read data (2'b11) accesses correct stored address in both DUT and golden model.

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
REQ_5	Verify read data operation (din[9:8] = 2'b11) outputs correct data and sets tx_valid.	Generate transaction pattern: write_addr → write_data → read_addr → read_data.	Cross coverage between din[9:8] and tx_valid=1. Check all 4 operation codes used.	Scoreboard compares dout vs dout_ref and tx_valid vs tx_valid_ref. Must match for all transactions.
REQ_6	Verify transaction ordering correctness.	Generate random valid/invalid sequences; include out-of-order operations.	Sequence coverage: valid order (WA→WD→RA→RD) vs invalid (WD→RA, etc.).	DUT and golden model should differ only for invalid sequences (expected error behavior).
REQ_7	Verify continuous read/write operations without reset in between.	Back-to-back random transactions without asserting reset.	Coverpoint for consecutive valid operations (rx_valid high multiple cycles).	DUT and golden model outputs must remain consistent under back-to-back operation.
REQ_8	Verify no output activity when rx_valid=0.	Randomize idle cycles with rx_valid=0.	Cross rx_valid=0 with tx_valid, expecting no active bins.	Scoreboard ensures no memory or output updates occur during idle cycles.

4. Assertion Table:

Feature	Assertion
Reset behavior: Whenever reset (rst_n) is deasserted, outputs dout and tx_valid must be 0.	`@(posedge clk) (!r_if.rst_n)
Input phase (address/write/read setup): For operations 00, 01, 10, tx_valid should remain low.	`@(posedge clk) disable iff(!r_if.rst_n) (r_if.din[9:8] inside {2'b00,2'b01,2'b10})
Output phase (read data operation): When operation is 2'b11 and rx_valid is high, tx_valid must assert in the next cycle.	`@(posedge clk) disable iff(!r_if.rst_n) (r_if.din[9:8]==2'b11 && r_if.rx_valid)
Write sequence ordering: Write data (01) must occur after write address (00).	`@(posedge clk) disable iff(!r_if.rst_n) (r_if.din[9:8]==2'b00)

Feature	Assertion
Read sequence ordering: Read data (11) must occur after read address (10).	`@(posedge clk) disable iff(!r_if.rst_n) (r_if.din[9:8]==2'b10)

5. Coverage Reports:

Coverage Report by instance with details

```
=====
=== Instance: /\top#dut /RAM_assertions_inst
=== Design Unit: work.RAM_assertions
=====
```

Assertion Coverage:

```
Assertions          5      5      0 100.00%
```

```
-----
Name                File(Line)                Failure  Pass
                        Count      Count
-----
```

```
/\top#dut /RAM_assertions_inst/assert__read_sequence_check
    RAM_assertions.sv(37)          0      1
/\top#dut /RAM_assertions_inst/assert__write_sequence_check
    RAM_assertions.sv(30)          0      1
/\top#dut /RAM_assertions_inst/assert__output_assert
    RAM_assertions.sv(23)          0      1
/\top#dut /RAM_assertions_inst/assert__input_assert
    RAM_assertions.sv(16)          0      1
/\top#dut /RAM_assertions_inst/assert__reset_assert
    RAM_assertions.sv(9)           0      1
```

Directive Coverage:

```
Directives          5      5      0 100.00%
```

DIRECTIVE COVERAGE:

```
-----
Name                Design Design  Lang File(Line)  Hits Status
                        Unit  UnitType
-----
/\top#dut /RAM_assertions_inst/cover__read_sequence_check
    RAM_assertions Verilog  SVA  RAM_assertions.sv(38)
                                2918 Covered
/\top#dut /RAM_assertions_inst/cover__write_sequence_check
```

```

                RAM_assertions Verilog SVA RAM_assertions.sv(31)
                        2996 Covered
/\top#dut /RAM_assertions_inst/cover__output_assert
                RAM_assertions Verilog SVA RAM_assertions.sv(24)
                        1462 Covered
/\top#dut /RAM_assertions_inst/cover__input_assert
                RAM_assertions Verilog SVA RAM_assertions.sv(17)
                        7737 Covered
/\top#dut /RAM_assertions_inst/cover__reset_assert
                RAM_assertions Verilog SVA RAM_assertions.sv(10)
                        33 Covered

```

DIRECTIVE COVERAGE:

```

-----
Name                Design Design  Lang File(Line)   Hits Status
                Unit  UnitType
-----
/\top#dut /RAM_assertions_inst/cover__read_sequence_check
                RAM_assertions Verilog SVA RAM_assertions.sv(38)
                        2918 Covered
/\top#dut /RAM_assertions_inst/cover__write_sequence_check
                RAM_assertions Verilog SVA RAM_assertions.sv(31)
                        2996 Covered
/\top#dut /RAM_assertions_inst/cover__output_assert
                RAM_assertions Verilog SVA RAM_assertions.sv(24)
                        1462 Covered
/\top#dut /RAM_assertions_inst/cover__input_assert
                RAM_assertions Verilog SVA RAM_assertions.sv(17)
                        7737 Covered
/\top#dut /RAM_assertions_inst/cover__reset_assert
                RAM_assertions Verilog SVA RAM_assertions.sv(10)
                        33 Covered

```

TOTAL DIRECTIVE COVERAGE: 100.00% COVERS: 5

ASSERTION RESULTS:

```

-----
Name                File(Line)                Failure    Pass
                Count          Count
-----
/\top#dut /RAM_assertions_inst/assert__read_sequence_check
                RAM_assertions.sv(37)          0          1
/\top#dut /RAM_assertions_inst/assert__write_sequence_check

```

```

RAM_assertions.sv(30)      0    1
/\top#dut /RAM_assertions_inst/assert__output_assert
RAM_assertions.sv(23)      0    1
/\top#dut /RAM_assertions_inst/assert__input_assert
RAM_assertions.sv(16)      0    1
/\top#dut /RAM_assertions_inst/assert__reset_assert
RAM_assertions.sv(9)       0    1

```

Total Coverage By Instance (filtered view): 100.00%

Screenshots:

1. UVM Report

```

# * Questa UVM Transaction Recording Turned ON.
# * recording_detail has been set.
# * To turn off, set 'recording_detail' to off:
# * uvm_config_db#(int) ::set(null, "", "recording_detail", 0);
# * uvm_config_db#(uvm_bitstream_t)::set(null, "", "recording_detail", 0);
# *****
# UVM_INFO RAM_test.sv(47) @ 10: uvm_test_top [run_phase] reset deasserted
# UVM_INFO RAM_test.sv(50) @ 110: uvm_test_top [run_phase] write_stimulus
# UVM_INFO RAM_test.sv(52) @ 1110: uvm_test_top [run_phase] end_stimulus
# UVM_INFO RAM_test.sv(55) @ 1210: uvm_test_top [run_phase] read_stimulus
# UVM_INFO RAM_test.sv(57) @ 2210: uvm_test_top [run_phase] end_stimulus
# UVM_INFO RAM_test.sv(60) @ 2310: uvm_test_top [run_phase] write_read_stimulus
# UVM_INFO RAM_test.sv(62) @ 102310: uvm_test_top [run_phase] end_stimulus
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 102310: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO scoreboard.sv(56) @ 102310: uvm_test_top.env.sb [report_phase] Total correct: 5115
# UVM_INFO scoreboard.sv(57) @ 102310: uvm_test_top.env.sb [report_phase] Total errors: 0
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 14
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
#
# ** Report counts by id
# [Questa UVM] 2
# [RNTST] 1
# [TEST_DONE] 1
# [report_phase] 2
# [run_phase] 8
#
# ** Note: $finish : C:/questasim64_2024.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
# Time: 102310 ns Iteration: 61 Instance: /top
# Break in Task uvm_pkg/uvm_root::run_test at C:/questasim64_2024.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh line 430

```

2. Coverage:

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comment
RAM_coverage_p...		100.00%							
TYPE cvr_grp		100.00%	100	100.00...				auto(0)	
CVP cvr_grp...		100.00%	100	100.00...					
CVP cvr_grp...		100.00%	100	100.00...					
CVP cvr_grp...		100.00%	100	100.00...					
CROSS cvr...		100.00%	100	100.00...					
CROSS cvr...		100.00%	100	100.00...					
INST VRA...		100.00%	100	100.00...					0
CVP cp...		100.00%	100	100.00...					
CVP cp...		100.00%	100	100.00...					
CVP cp...		100.00%	100	100.00...					
CROSS...		100.00%	100	100.00...					
CROSS...		100.00%	100	100.00...					

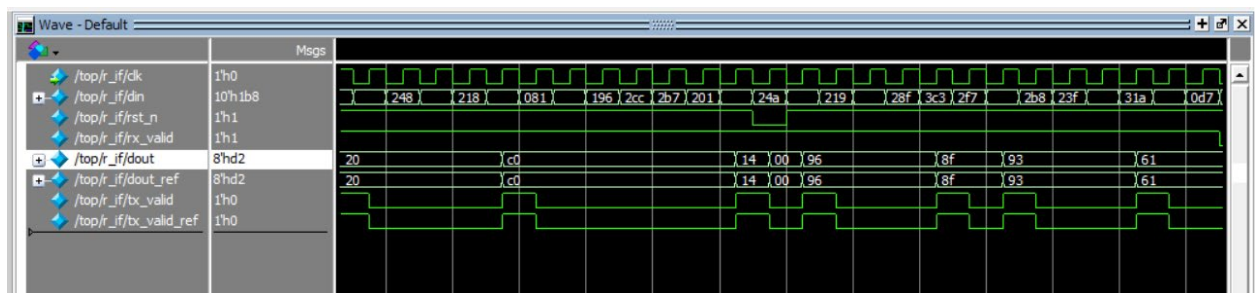
3. Assertions:

Cover Directives													
Lang	Enab	Name	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Included	Memory	Peak Memory	Peak Memory Time
SVA	✓	/top/dut/RAM_assertions_inst/cover_read_sequen...	Off	2918	1	Unli...	1	100%		✓	0	0	0 i
SVA	✓	/top/dut/RAM_assertions_inst/cover_write_seque...	Off	2996	1	Unli...	1	100%		✓	0	0	0 i
SVA	✓	/top/dut/RAM_assertions_inst/cover_output_asse...	Off	1462	1	Unli...	1	100%		✓	0	0	0 i
SVA	✓	/top/dut/RAM_assertions_inst/cover_input_assert	Off	7737	1	Unli...	1	100%		✓	0	0	0 i
SVA	✓	/top/dut/RAM_assertions_inst/cover_reset_assert	Off	33	1	Unli...	1	100%		✓	0	0	0 i

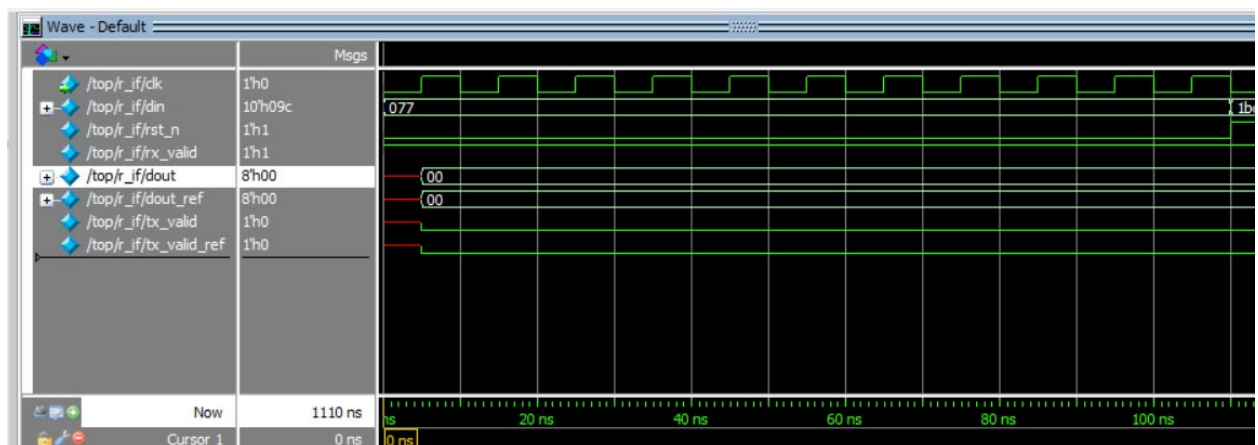
Assertions											
Name	Assertion Type	Language	Enable	Failure Count	Pass Count	Active Count	Memory	Peak Memory	Peak Memory	Peak Memory	Peak Memory Time
/uvm_pkg::uvm_reg_map::do_write/#ublk#215181159#1731/immed__1735	Immediate	SVA	on	0	0	-	-	-	-	-	-
/uvm_pkg::uvm_reg_map::do_read/#ublk#215181159#1771/immed__1775	Immediate	SVA	on	0	0	-	-	-	-	-	-
/write_read_sequence_pkg::RAM_write_read_sequence::body/#ublk#1521613...	Immediate	SVA	on	0	1	-	-	-	-	-	-
/read_only_sequence_pkg::RAM_read_only_sequence::body/#ublk#14091627...	Immediate	SVA	on	0	1	-	-	-	-	-	-
/write_only_sequence_pkg::RAM_write_only_sequence::body/#ublk#15486563...	Immediate	SVA	on	0	1	-	-	-	-	-	-
/top/dut/RAM_assertions_inst/assert_reset_assert	Concurrent	SVA	on	0	1	-	0B	0B	0B	0 ns	-
/top/dut/RAM_assertions_inst/assert_input_assert	Concurrent	SVA	on	0	1	-	0B	0B	0B	0 ns	-
/top/dut/RAM_assertions_inst/assert_output_assert	Concurrent	SVA	on	0	1	-	0B	0B	0B	0 ns	-
/top/dut/RAM_assertions_inst/assert_write_sequence_check	Concurrent	SVA	on	0	1	-	0B	0B	0B	0 ns	-
/top/dut/RAM_assertions_inst/assert_read_sequence_check	Concurrent	SVA	on	0	1	-	0B	0B	0B	0 ns	-

4. Waveform

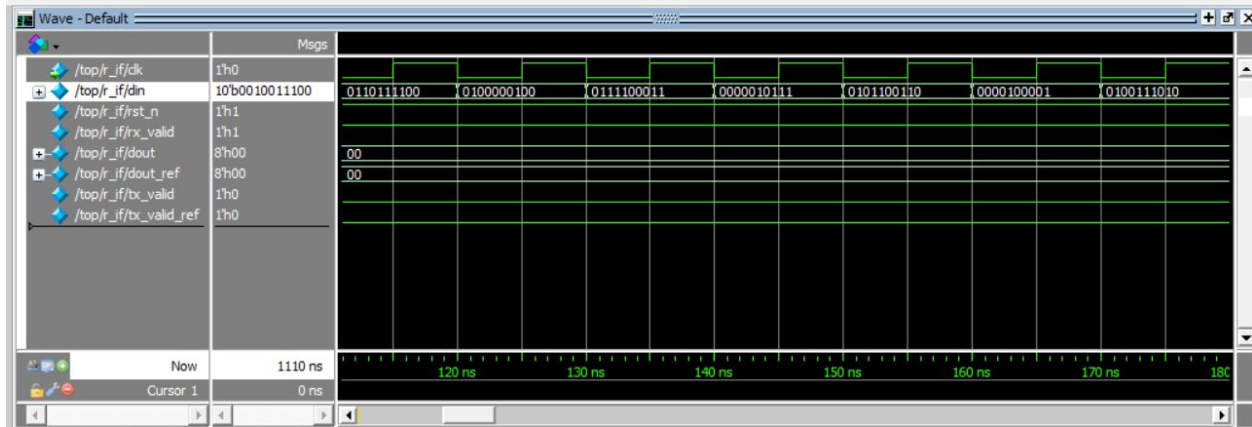
a. Random snippet



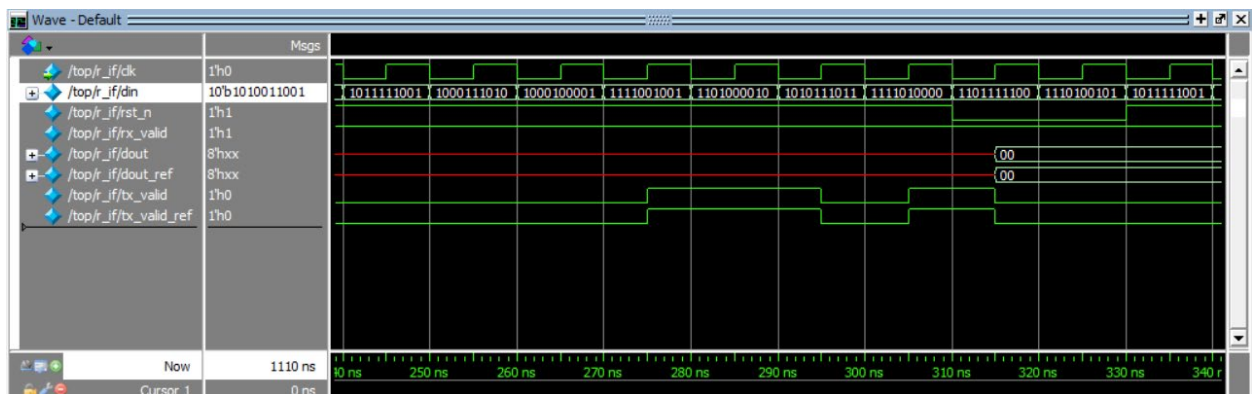
b. Reset sequence



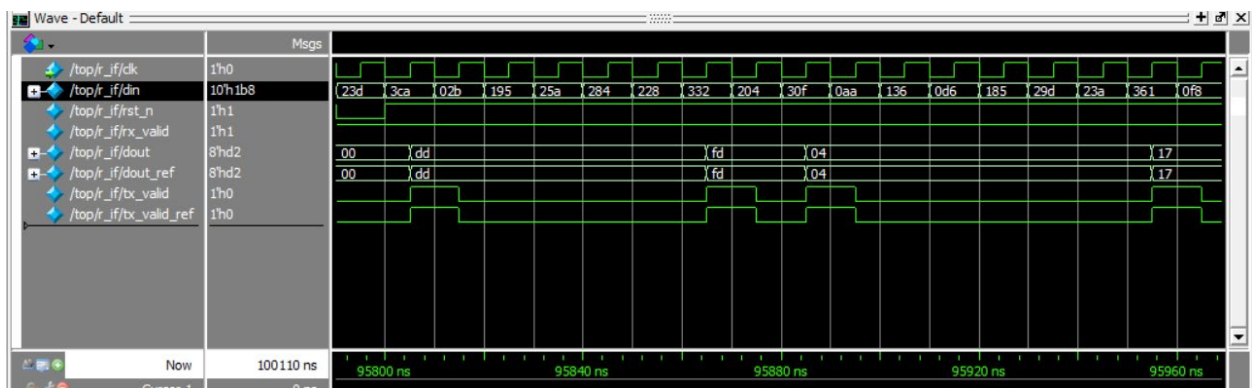
c. Write Only



d. Read Only



e. Write Read



Codes:

1. Run.do

```
vlib work
vlog -f src_files.list
vsim -voptargs=+acc work.top -classdebug -uvmcontrol=all
add wave /top/r_if/*
```

```
run -all
```

2. Src_file.list

```
RAM_coverage.sv  
scoreboard.sv  
monitor.sv  
agent.sv  
RAM_seq_item.sv  
RAM_sequencer.sv  
reset_sequence.sv  
write_only_sequence.sv  
read_only_sequence.sv  
write_read_sequence.sv  
RAM_config.sv  
RAM_driver.sv  
RAM_env.sv  
RAM_test.sv  
RAM.sv  
SinglePort_SRAM.v  
RAM_if.sv  
RAM_assertions.sv  
top.sv
```

3. RAM coverage

```
package RAM_coverage_pkg;  
  
import uvm_pkg::*;  
import RAM_seq_item_pkg::*;  
import RAM_sequencer_pkg::*;  
import monitor_pkg::*;  
`include "uvm_macros.svh"  
  
class RAM_coverage extends uvm_component;  
  
    `uvm_component_utils(RAM_coverage)  
    uvm_analysis_export #(RAM_seq_item) cov_export;  
    uvm_tlm_analysis_fifo #(RAM_seq_item) cov_fifo;  
    RAM_seq_item seq_item_cov;  
  
    covergroup cvr_grp;  
        cp_op : coverpoint seq_item_cov.din[9:8] {  
            bins wr_addr = {2'b00};  
            bins wr_data = {2'b01};  
            bins rd_addr = {2'b10};  
            bins rd_data = {2'b11};  
        }  
    endcovergroup  
endclass
```



```

        // transition bins
        bins wr_addr_to_wr_data = (2'b00 => 2'b01);
        bins rd_addr_to_rd_data = (2'b10 => 2'b11);
        bins wr_rd_sequence = (2'b00 => 2'b01 => 2'b10 => 2'b11);
    }

    cp_rx : coverpoint seq_item_cov.rx_valid {
        bins r_valid_low  = {1'b0};
        bins r_valid_high = {1'b1};
    }

    cp_tx : coverpoint seq_item_cov.tx_valid {
        bins t_valid_low  = {1'b0};
        bins t_valid_high = {1'b1};
    }

    // cross: din[9:8] x rx_valid (only need the rx_valid high in the cross)
    cross_op_rx : cross cp_op, cp_rx {
        bins cross_when_valid = binsof(cp_op) && binsof(cp_rx.r_valid_high);
    }

    // cross: when din == read_data (2'b11) cross tx_valid==1
    cross_rd_tx : cross cp_op, cp_tx {
        bins rddata_txvalid = binsof(cp_op) && binsof(cp_tx.t_valid_high);
    }
endgroup

function new(string name = "RAM_coverage", uvm_component parent = null);
    super.new(name, parent);
    cvr_grp = new();
endfunction

function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    cov_export = new("cov_export", this);
    cov_fifo = new("cov_fifo", this);
endfunction

function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    cov_export.connect(cov_fifo.analysis_export);
endfunction

task run_phase(uvm_phase phase);

```

```

        super.run_phase(phase);
        forever begin
            cov_fifo.get(seq_item_cov);
            cvr_grp.sample();
        end
    endtask
endclass

endpackage

```

4. Scoreboard

```

package RAM_scoreboard_pkg;

import uvm_pkg::*;
import RAM_seq_item_pkg::*;
import RAM_sequencer_pkg::*;
import monitor_pkg::*;
`include "uvm_macros.svh"

class RAM_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(RAM_scoreboard)
    uvm_analysis_export #(RAM_seq_item) sb_export;
    uvm_tlm_analysis_fifo #(RAM_seq_item) sb_fifo;
    RAM_seq_item seq_item_sb;
    logic signed [5:0] out_ref;

    int error_count = 0;
    int correct_count = 0;

    function new(string name = "RAM_scoreboard", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        sb_export = new("sb_export", this);
        sb_fifo = new("sb_fifo", this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        sb_export.connect(sb_fifo.analysis_export);
    endfunction

```

```

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            sb_fifo.get(seq_item_sb);
            ref_model(seq_item_sb);

            if (seq_item_sb.dout != seq_item_sb.dout_ref || seq_item_sb.tx_valid
!= seq_item_sb.tx_valid_ref) begin
                `uvm_info("run_phase", $sformatf("Comparison unsuccessful,
transaction: %s",
                    seq_item_sb.convert2string()), UVM_HIGH)
                error_count++;
            end else begin
                correct_count++;
            end
        end
    endtask

    task ref_model(RAM_seq_item seq_item_chk);

    endtask

    function void report_phase(uvm_phase phase);
        super.report_phase(phase);
        `uvm_info("report_phase", $sformatf("Total correct: %0d", correct_count),
UVM_MEDIUM);
        `uvm_info("report_phase", $sformatf("Total errors: %0d", error_count),
UVM_MEDIUM);
    endfunction

endclass

endpackage

```

5. Monitor

```

package monitor_pkg;

import uvm_pkg::*;
import RAM_seq_item_pkg::*;
`include "uvm_macros.svh"

class monitor extends uvm_monitor;

```

```

`uvm_component_utils(monitor)
virtual RAM_if RAM_vif;
RAM_seq_item rsp_seq_item;
uvm_analysis_port #(RAM_seq_item) mon_ap;

function new(string name = "monitor", uvm_component parent = null);
    super.new(name, parent);
endfunction

function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    mon_ap = new("mon_ap", this);
endfunction

task run_phase(uvm_phase phase);
    super.run_phase(phase);
    forever begin
        rsp_seq_item = RAM_seq_item::type_id::create("rsp_seq_item");
        @(negedge RAM_vif.clk);
        rsp_seq_item.rst_n = RAM_vif.rst_n;
        rsp_seq_item.din = RAM_vif.din;
        rsp_seq_item.rx_valid = RAM_vif.rx_valid;
        @(negedge RAM_vif.clk);
        rsp_seq_item.dout = RAM_vif.dout;
        rsp_seq_item.tx_valid = RAM_vif.tx_valid;
        rsp_seq_item.dout_ref = RAM_vif.dout_ref;
        rsp_seq_item.tx_valid_ref = RAM_vif.tx_valid_ref;
        //if (RAM_vif.rst_n) begin
            mon_ap.write(rsp_seq_item);
        //end
        `uvm_info("run_phase", rsp_seq_item.convert2string_stimulus(),
UVM_HIGH);
    end
endtask

endclass

endpackage

```

6. Agent

7. Sequence Item

```

package RAM_seq_item_pkg;
import uvm_pkg::*;
`include "uvm_macros.svh"

class RAM_seq_item extends uvm_sequence_item;
    `uvm_object_utils(RAM_seq_item)

    rand logic [9:0] din;
    rand logic rst_n, rx_valid;
    logic [7:0] dout, dout_ref;
    logic tx_valid, tx_valid_ref;

    function new(string name = "RAM_seq_item");
        super.new(name);
    endfunction

    function string convert2string();
        return $sformatf(
            "rst_n=%0b, rx_valid=%b, din=%0b, dout=%b, tx_valid=%b, dout_ref=%b,
tx_valid_ref=%b",
            rst_n, rx_valid, din, dout, tx_valid, dout_ref, tx_valid_ref
        );
    endfunction

    function string convert2string_stimulus();
        return $sformatf(
            "rst_n=%0b, rx_valid=%b, din=%0b",
            rst_n, rx_valid, din
        );
    endfunction

    constraint c_reset_low_prob { rst_n dist { 0 := 5, 1 := 95 };}
    constraint c_rx_valid_low_prob { rx_valid dist { 0 := 5, 1 := 95 };}
    //write only, read only and read write
    //will be in the sequence file
endclass
endpackage

```

8. Sequencer

```

package RAM_sequencer_pkg;

import uvm_pkg::*;
`include "uvm_macros.svh"
import RAM_seq_item_pkg::*;

```

```

class RAM_sequencer extends uvm_sequencer #(RAM_seq_item);
    `uvm_component_utils(RAM_sequencer)

    function new(string name = "RAM_sequencer", uvm_component parent = null);
        super.new(name, parent);
    endfunction

endclass

endpackage

```

9. Reset Sequence

```

package reset_sequence_pkg;

import uvm_pkg::*;
`include "uvm_macros.svh"
import RAM_seq_item_pkg::*;

class RAM_reset_sequence extends uvm_sequence #(RAM_seq_item);
    `uvm_object_utils(RAM_reset_sequence)
    RAM_seq_item seq_item;

    function new(string name = "RAM_reset_sequence");
        super.new(name);
    endfunction

    task body;
        seq_item = RAM_seq_item::type_id::create("seq_item");
        start_item(seq_item);
        seq_item.rst_n = 0;
        seq_item.din = 10'b00_0111_0111;
        seq_item.rx_valid = 1;
        finish_item(seq_item);
    endtask

endclass

endpackage

```

10. Write Only Seq

```

package write_only_sequence_pkg;

```

```

import uvm_pkg::*;
`include "uvm_macros.svh"
import RAM_seq_item_pkg::*;

class RAM_write_only_sequence extends uvm_sequence #(RAM_seq_item);
    `uvm_object_utils(RAM_write_only_sequence)
    RAM_seq_item seq_item;

    bit [1:0] prev_op;

    function new(string name = "RAM_write_only_sequence");
        super.new(name);
        prev_op = 2'b10;
    endfunction

    task body;
        seq_item = RAM_seq_item::type_id::create("seq_item");
        repeat(100) begin
            start_item(seq_item);
            assert(seq_item.randomize() with {
                din[9:8] inside {2'b00, 2'b01};
                if (prev_op == 2'b00)
                    din[9:8] inside {2'b01, 2'b00};
                else if (prev_op == 2'b01)
                    din[9:8] inside {2'b01, 2'b00};
            })
            finish_item(seq_item);
            prev_op = seq_item.din[9:8];
        end
    endtask

endclass

endpackage

```

11. Read Only Seq

```

package read_only_sequence_pkg;

import uvm_pkg::*;
`include "uvm_macros.svh"
import RAM_seq_item_pkg::*;

```

```

class RAM_read_only_sequence extends uvm_sequence #(RAM_seq_item);
  `uvm_object_utils(RAM_read_only_sequence)
  RAM_seq_item seq_item;

  bit [1:0] prev_op;

  function new(string name = "RAM_read_only_sequence");
    super.new(name);
    prev_op = 2'b10;
  endfunction

  task body;
    seq_item = RAM_seq_item::type_id::create("seq_item");
    repeat(100) begin
      start_item(seq_item);
      assert(seq_item.randomize() with {
        din[9:8] inside {2'b10, 2'b11};
        if (prev_op == 2'b10)
          din[9:8] inside {2'b10, 2'b11};
        else if (prev_op == 2'b11)
          din[9:8] inside {2'b10, 2'b11};
      })
      finish_item(seq_item);
      prev_op = seq_item.din[9:8];
    end
  endtask

endclass

endpackage

```

12. Write Read Seq

```

package write_read_sequence_pkg;

import uvm_pkg::*;
`include "uvm_macros.svh"
import RAM_seq_item_pkg::*;

class RAM_write_read_sequence extends uvm_sequence #(RAM_seq_item);
  `uvm_object_utils(RAM_write_read_sequence)
  RAM_seq_item seq_item;

  bit [1:0] prev_op;

```



```

localparam bit [1:0] WRITE_ADDR = 2'b00;
localparam bit [1:0] WRITE_DATA = 2'b01;
localparam bit [1:0] READ_ADDR  = 2'b10;
localparam bit [1:0] READ_DATA  = 2'b11;

function new(string name = "RAM_write_read_sequence");
    super.new(name);
    prev_op = WRITE_ADDR;
endfunction

task body;
    seq_item = RAM_seq_item::type_id::create("seq_item");
    repeat(10000) begin
        start_item(seq_item);
        assert(seq_item.randomize() with {
            (prev_op == WRITE_ADDR) -> (din[9:8] inside {WRITE_ADDR,
WRITE_DATA});
            (prev_op == WRITE_DATA) -> din[9:8] dist {READ_ADDR := 60,
WRITE_ADDR := 40};
            (prev_op == READ_ADDR)  -> (din[9:8] inside {READ_ADDR,
READ_DATA});
            (prev_op == READ_DATA)  -> din[9:8] dist {WRITE_ADDR := 60,
READ_ADDR := 40};
        });
        finish_item(seq_item);
        prev_op = seq_item.din[9:8];
    end
endtask
endclass
endpackage

```

13. Config

```

package RAM_config_pkg;

import uvm_pkg::*;
`include "uvm_macros.svh"

class RAM_config_obj extends uvm_object;

    `uvm_object_utils(RAM_config_obj)
    virtual RAM_if RAM_vif;

    function new(string name = "RAM_config_obj");

```

```

        super.new(name);
    endfunction

endclass

endpackage

```

14. Driver

```

package RAM_driver_pkg;

import uvm_pkg::*;
import RAM_seq_item_pkg::*;
`include "uvm_macros.svh"

class RAM_driver extends uvm_driver #(RAM_seq_item);
    `uvm_component_utils(RAM_driver)
    virtual RAM_vif RAM_vif;
    RAM_seq_item stim_seq_item;

    function new(string name = "RAM_driver", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            stim_seq_item = RAM_seq_item::type_id::create("stim_seq_item");
            seq_item_port.get_next_item(stim_seq_item);
            RAM_vif.rst_n = stim_seq_item.rst_n;
            RAM_vif.din = stim_seq_item.din;
            RAM_vif.rx_valid = stim_seq_item.rx_valid;
            @(negedge RAM_vif.clk);
            seq_item_port.item_done();
            `uvm_info("run_phase", stim_seq_item.convert2string_stimulus(),
UVM_HIGH);
        end
    endtask

endclass

endpackage

```

15. Environment

```
package RAM_env_pkg;

import uvm_pkg::*;
import RAM_coverage_pkg::*;
import RAM_scoreboard_pkg::*;
import agent_pkg::*;

`include "uvm_macros.svh"

class RAM_env extends uvm_env;
    `uvm_component_utils(RAM_env)

    //RAM_driver driver;
    agent agt;
    RAM_scoreboard sb;
    RAM_coverage cov;

    function new(string name = "RAM_env", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase (uvm_phase phase);
        super.build_phase(phase);
        agt = agent::type_id::create("agt", this);
        sb = RAM_scoreboard::type_id::create("sb", this);
        cov = RAM_coverage::type_id::create("cov", this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        agt.agt_ap.connect(sb.sb_export);
        agt.agt_ap.connect(cov.cov_export);
    endfunction
endclass
endpackage
```

16. Test

```
package RAM_test_pkg;
import RAM_env_pkg::*;
import RAM_config_pkg::*;
import reset_sequence_pkg::*;
import write_only_sequence_pkg::*;
```

```

import read_only_sequence_pkg::*;
import write_read_sequence_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class RAM_test extends uvm_test;

    `uvm_component_utils(RAM_test)

    RAM_env env;
    RAM_config_obj RAM_cfg;
    virtual RAM_if RAM_vif;
    RAM_reset_sequence reset_seq;
    RAM_write_only_sequence write_seq;
    RAM_read_only_sequence read_seq;
    RAM_write_read_sequence write_read_seq;

    function new(string name = "RAM_test", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        env = RAM_env::type_id::create("env", this);
        RAM_cfg = RAM_config_obj::type_id::create("RAM_cfg");
        reset_seq = RAM_reset_sequence::type_id::create("reset_seq");
        write_seq = RAM_write_only_sequence::type_id::create("write_seq");
        read_seq = RAM_read_only_sequence::type_id::create("read_seq");
        write_read_seq =
RAM_write_read_sequence::type_id::create("write_read_seq");

        if(!uvm_config_db #(virtual RAM_if)::get(this, "", "RAM_IF",
RAM_cfg.RAM_vif))
            `uvm_fatal("build phase", "Unabble to get V_IF");
        uvm_config_db #(RAM_config_obj)::set(this, "*", "CFG", RAM_cfg);

    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        phase.raise_objection(this);
        `uvm_info("run_phase", "reset asserted", UVM_LOW);
        reset_seq.start(env.agt.sqr); //since sqr is in agent NOT environment
        `uvm_info("run_phase", "reset deasserted", UVM_LOW);

```

```

#100;
`uvm_info("run_phase", "write_stimulus", UVM_LOW);
write_seq.start(env.agt.sqr);
`uvm_info("run_phase", "end stimulus", UVM_LOW);

#100;
`uvm_info("run_phase", "read_stimulus", UVM_LOW);
read_seq.start(env.agt.sqr);
`uvm_info("run_phase", "end stimulus", UVM_LOW);

#100;
`uvm_info("run_phase", "write_read_stimulus", UVM_LOW);
write_read_seq.start(env.agt.sqr);
`uvm_info("run_phase", "end stimulus", UVM_LOW);

    phase.drop_objection(this);
endtask

endclass: RAM_test
endpackage

```

17. RAM

```

module RAM (RAM_if r_if);

reg [7:0] MEM [255:0];

reg [7:0] Rd_Addr, Wr_Addr;

always @(posedge r_if.clk) begin
    if (~r_if.rst_n) begin
        r_if.dout <= 0;
        r_if.tx_valid <= 0;
        Rd_Addr <= 0;
        Wr_Addr <= 0;
    end
    else
        if (r_if.rx_valid) begin
            case (r_if.din[9:8])
                2'b00 : Wr_Addr <= r_if.din[7:0];
                2'b01 : MEM[Wr_Addr] <= r_if.din[7:0];
                2'b10 : Rd_Addr <= r_if.din[7:0];
                2'b11 : r_if.dout <= MEM[Rd_Addr]; //changed to read address
                default : r_if.dout <= 0;
            endcase
        end
    end
end

```



```

        data <= din[7:0];
        mem[address] <= din[7:0]; // Write to memory
        tx_valid <= 1'b0;
    end
    2'b10: begin
        address <= din[7:0]; // Set address for reading
        tx_valid <= 1'b0;
    end
    2'b11: begin
        dout <= mem[address]; // Read from memory
        tx_valid <= 1'b1;
    end
    default: tx_valid <= 1'b0;
endcase
end else begin
    tx_valid <= 1'b0;
end
end
end
endmodule

```

19. Interface

```

interface RAM_if(
    input clk
);
    logic [9:0] din;
    logic rst_n, rx_valid;
    logic [7:0] dout, dout_ref;
    logic tx_valid, tx_valid_ref;
endinterface: RAM_if

```

20. Assertions

```

module RAM_assertions(
    RAM_if r_if
);

    property reset_assert;
        @(posedge r_if.clk) disable iff(r_if.rst_n)
            (!r_if.rst_n) | => (r_if.dout == 0) && (r_if.tx_valid == 0);
    endproperty
    assert property(reset_assert);
    cover property (reset_assert);

```

```

    property input_assert;
        @(posedge r_if.clk) disable iff(!r_if.rst_n)
            (r_if.din[9:8] inside {2'b00, 2'b01, 2'b10}) | => (r_if.tx_valid ==
0);
    endproperty
    assert property(input_assert);
    cover property (input_assert);

    property output_assert;
        @(posedge r_if.clk) disable iff(!r_if.rst_n)
            (r_if.din[9:8] inside {2'b11} && r_if.rx_valid) | => (r_if.tx_valid
== 1);
    endproperty
    assert property(output_assert);
    cover property (output_assert);

    property write_sequence_check;
        @(posedge r_if.clk) disable iff (!r_if.rst_n)
            (r_if.din[9:8] == 2'b00) | -> ##[1:$] (r_if.din[9:8] == 2'b01);
    endproperty
    assert property (write_sequence_check);
    cover property (write_sequence_check);

    property read_sequence_check;
        @(posedge r_if.clk) disable iff (!r_if.rst_n)
            (r_if.din[9:8] == 2'b10) | -> ##[1:$] (r_if.din[9:8] == 2'b11);
    endproperty
    assert property (read_sequence_check);
    cover property (read_sequence_check);

endmodule

```

21. Top

```

import uvm_pkg::*;
import RAM_test_pkg::*;

`include "uvm_macros.svh"

module top();

    bit clk;

```



```

initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

RAM_if r_if(clk);
RAM dut (.r_if(r_if));
SinglePort_SRAM golden_model (
    .clk(clk),
    .rst_n(r_if.rst_n),
    .rx_valid(r_if.rx_valid),
    .din(r_if.din),
    .dout(r_if.dout_ref),
    .tx_valid(r_if.tx_valid_ref)
);
bind RAM RAM_assertions RAM_assertions_inst (.r_if(r_if));

initial begin
    uvm_config_db#(virtual RAM_if)::set(null, "uvm_test_top", "RAM_IF",
r_if);
    run_test("RAM_test");
end

endmodule

```